Part A

1. RGB to Grayscale and Binary Conversion:
    - Convert an RGB image to a grayscale image.
    - Threshold the grayscale image to create a binary image.
2. Splitting RGB Channels:
    - Extract and display the Red, Green, and Blue channels separately from an RGB image.
3. RGB to Different Color Space Conversion:
    - Convert an RGB image to other color spaces.
    - Display the individual channels of the new color spaces.
4. Downsampling and Upsampling:
    - Downsample the original RGB image to reduce its resolution.
    - Upsample the downsampled image back to its original size.
    - Compare the original and upsampled images.

Part B

1. Image Negative

Compute the negative of a grayscale image. Formula: $s = L - 1 - r$, where L is the intensity range and r is the pixel intensity.

2. Log Transformation

Apply log transformation to enhance dark regions of an image. Formula: $s = c \cdot log(1 + r)$ where c is a scaling constant.

3. Image Whitening

Normalize pixel intensities to have a mean of 0 and standard deviation of 1.

4. Contrast Stretching

Stretch the intensity values of a gray-scale image to span the entire range [0, 255].

5. Histogram Equalization

Perform histogram equalization on an image enhance its contrast. Use OpenCV's *cv2.equalizeHist()* function.

Part C

1. Low-Pass Filtering

Apply an average filter (3×3 and 5×5 kernel) on an image and observe the degree of smoothing. Apply a Gaussian filter (3×3 and 5×5 kernel) and compare its effect with the average filter.

2. Median Filtering

Add impulse noise (salt-and-pepper) to an image and apply a median filter (3×3 and 5×5 kernel). Compare the performance of the median filter with the Gaussian filter and average filter for noise removal.

3. High-Pass Filtering

Apply a $1D$ kernel HPF on an image (e.g., Sobel or Prewitt). Observe the enhancement of edges. Apply the Laplacian filter and compare its results with the $1D$ kernel. Combine the original image with the result of the Laplacian filter to create a sharpened image.

Part D

1. Frequency Spectrum Visualization

Load an image and convert it to grayscale. Perform a 2D Fourier Transform using OpenCV's cv2.dft. Visualize the frequency spectrum using a log scale.

2. Low-Pass Filtering (LPF)

Apply an ideal low-pass filter by masking out high-frequency components beyond a certain radius. Implement a Gaussian low-pass filter in the frequency domain. Compare the effects of ideal and Gaussian LPFs.

3. High-Pass Filtering

Use Gaussian HPF to detect high frequency components of an image. Compare the result with spatial-domain Gaussian High Pass filtering for various kernel sizes.

4. Notch Reject Filtering

Add periodic noise to an image (e.g., sine waves of a fixed frequency). Design a notch reject filter to remove the periodic noise. Identify the noise's frequency components and create corresponding notch filters. Apply the notch filter and visualize the enhanced image.

Part E

1. Gradient-Based Edge Detection

Apply Sobel and Prewitt operators on a grayscale image and visualize the gradient magnitude and direction. Compare the performance of Sobel and Prewitt edge detectors.

2. Canny Edge Detection

Apply the Canny Edge Detector step by step on an image and experiment with different values of Gaussian kernel size and threshold parameters.

3. Marr-Hildreth Edge Detection

Apply the Marr-Hildreth edge detector (Laplacian of Gaussian) on an image and visualize the zero-crossing map. Compare the Marr-Hildreth detector with the Canny edge detector.

Part F

1. Harris Corner Detector
Apply the Harris Corner Detector to images of text, buildings, and natural scenes, captured by your phone. Experiment with different values of the Harris response threshold and Compare the performance.

Part G

Scale-Invariant Feature Transform

- Implement SIFT feature detection using OpenCV and visualize the detected keypoints on an image.
- Compare the number of keypoints detected by SIFT on different image resolutions (low vs. high resolution).
- Apply rotation, scaling, and illumination changes to an image and compare how SIFT keypoints are affected.
- Use SIFT to detect features in two images of the same object taken from different viewpoints. Count the number of matches between the two images.

Part H

Co-occurrence matrix

Capture or download two different textured images (e.g., brick wall vs. cloth fabric) and Compute the Co-occurrence matrix of the images for different position operators.

Extract the texture features such as contrast, correlation, energy, entropy, and homogeneity for both images. Compare texture feature values of both images.